Near-optimal separation of ordered from unordered set-multilinear algebraic branching programs

Prerona Chatterjee, Deepanshu Kush, Shubhangi Saraf, Amir Shpilka

doi:10.4230/LIPIcs.CCC.2024.20; CC BY-NC-SA4.0

Editorial note. Original source: Prerona Chatterjee, Deepanshu Kush, Shubhangi Saraf and Amir Shpilka; arXiv:2312.15874v3 (2024-02-19), https://arxiv.org/abs/2312.15874v3. License: Creative Commons Attribution-NonCommercial-ShareAlike4.0, https://creativecommons.org/licenses/by-nc-sa/4.0/.

Editorial note. Non-commercial research collection. Changes: complete author theorem/proof/context excerpts and original macros/bibliography in standalone layout; unrelated sections and frontmatter omitted. No mathematical corrections or additional proofs. Adapted presentation retains CC BY-NC-SA4.0 including NC/SA restrictions.

Allowed standard prerequisites (editorial).

- Nisan ordered-ABP rank-width lower bound. Author Lemma lem:nisan-abpsize explicitly cites Nisan91 and states the exact relative-rank form inside the full proof.
- Standard relative-rank multiplicativity and inner-product rank gadget. Author Claim clm:rk-props and cited Observation obs:ip-full-rank; exact statements retained.

Source reference. Original section 1.3: number-theory/sources/ccc-2024-20/native/main.tex, source lines 150–150. Optional introductory pointer to results; counted full target statements and proofs are included.

An *algebraic circuit* over a field $\mathbb{F}$ for a multivariate polynomial $P(x_1, \ldots, x_N)$ is a directed acyclic graph (DAG) whose internal vertices (called gates) are labeled as either + (sum) or × (product), and leaves (vertices of in-degree zero) are labeled by the variables x_i or constants from $\mathbb{F}$. A special output gate (the root of the DAG) represents the polynomial P. If the DAG happens to be a tree, such a resulting circuit is called an *algebraic formula*. The size of a circuit or formula is the number of nodes in the DAG. We also consider the product-depth of the circuit, which is the maximum number of product gates on a root-to-leaf path. The class VP (respectively, VF) is then defined to be the collection of all polynomials having at most polynomially large degree which can be computed by polynomial-sized circuits (respectively, formulas).

An *algebraic branching program* (ABP) is a layered DAG with two special nodes in it: a start-node and an end-node. All edges of the ABP go from

layer $\ell - 1$ to layer ℓ for some ℓ (say start-node is the unique node in layer 0 and end-node is the unique node in the last layer) and are labeled by a linear polynomial. Every directed path γ from start-node to end-node computes the monomial P_γ, which is the product of all labels on the path γ. The ABP computes the polynomial $P = \sum_\gamma P_\gamma$, where the sum is over all paths γ from start-node to end-node. Its size is simply the number of nodes in the DAG, its *depth* is the length of the longest path from the start-node to the end-node, and *width* is the maximum number of nodes in any layer. The class VBP is then defined to be the collection of all polynomials (with polynomially-bounded degree) which can be computed by polynomial-sized branching programs. ABPs are known to be of intermediate complexity between formulas and circuits; in other words, we know the inclusions $\mathsf{VF} \subseteq \mathsf{VBP} \subseteq \mathsf{VP} \subseteq \mathsf{VNP}$.

A polynomial is said to be *homogeneous* if each monomial has the same total degree and *multilinear* if every variable occurs at most once in any monomial. Now, suppose that the underlying variable set is partitioned into d sets $X_1, \ldots, X_d$. Then the polynomial is said to be *set-multilinear* with respect to this variable partition if each monomial in P has *exactly* one variable from each set. Note that a set-multilinear polynomial is both multilinear and homogeneous, and has degree precisely d if it is set-multilinear over d sets. Next, we define different models of computation corresponding to these variants of polynomials classes. An algebraic formula/branching program/circuit is set-multilinear with respect to a variable partition $(X_1, \ldots, X_d)$ if each internal node in the formula/branching program/circuit computes a set-multilinear polynomial.[1] Multilinear and homogeneous formulas/branching programs/circuits are defined analogously.

Definition 0.1 (Ordered smABP)**.** Given a variable partition $(X_1, \ldots, X_d)$, we say that a set-multilinear branching program of depth d is said to be *ordered* with respect to an ordering (or permutation) $\sigma \in S_d$ if for each $\ell \in [d]$, all edges of the ABP from layer $\ell - 1$ to layer ℓ are labeled using a linear form over the variables in $X_{\sigma(\ell)}$. It is simply said to be *ordered* if there *exists* an ordering σ such that it is ordered with respect to σ.

At this point, it is essential to take note of the terminology in this context: in this paper, a general (or "unordered") set-multilinear branching program refers to an ABP for which each internal node computes a polynomial that is set-multilinear with respect to *some* subset of the global partition, whereas an *ordered* set-multilinear branching program is more specialized and has the property that any two nodes in the same layer compute polynomials that are set-multilinear with respect to the *same* partition.

Definition 0.2 ($\sum$smABP)**.** Given a polynomial $P(X)$ that is set-multilinear with respect to the variable partition $X = (X_1, \ldots, X_d)$, we say that $\sum_{i=1}^{t} A_i$ is a $\sum$smABP computing P if indeed $\sum_{i=1}^{t} A_i(X) = P(X)$, and each A_i is an ordered set-multilinear branching program i.e., each A_i is ordered with respect to some $\sigma_i \in S_d$. We call t (i.e., the number of summands in a $\sum$smABP) its *support size* and define its *max-width* and *total-width* to be the maximum over the width of each A_i and the sum of the width of each A_i, respectively.

[1] Of course, a non-root node need not be set-multilinear with respect to the *entire* variable partition. Nevertheless, here we demand that it must be set-multilinear with respect to some *subset* of the collection $\{X_1, \ldots, X_d\}$.

Theorem 0.3 (Near-Optimal Separation between Ordered and Unordered smABPs)**.** *There is a polynomial $G_{n,d}(X)$ which is set-multilinear over the variable partition $X = (X_1, \ldots, X_d)$ with each $|X_i| \leq n$, and such that:*

- *it has a set-multilinear branching program of size* $\mathrm{poly}(n, d)$*,*
- *but any ordered set-multilinear branching program computing $G_{n,d}$ requires width $n^{\Omega(d)}$.*

1 Preliminaries

1.1 Relative Rank and its Properties

We first describe the notation that we need to define the measures that we use to prove our results described in Section 1.3. Instead of directly working with the rank of the partial derivative matrix, we work with the following normalized form.

Definition 1.1. Let $w = (w_1, w_2, \ldots, w_d)$ be a tuple (or word) of non-zero real numbers. For a subset $S \subseteq [t]$, we shall refer to the sum $\sum_{i \in S} w_i$ by w_S, and by $w|_S$, we will refer to the tuple obtained by considering only the elements of w that are indexed by S. Given a word $w = (w_1, \ldots, w_d)$, we denote by $\overline{X}(w)$ a tuple of d sets of variables $(X(w_1), \ldots, X(w_d))$ where $|X(w_i)| = 2^{|w_i|}$.[2] We denote by $\mathbb{F}_{\mathrm{sm}}[\mathcal{T}]$ the set of set-multilinear polynomials over the tuple of sets of variables $\mathcal{T}$.

Definition 1.2 (Relative Rank Measure of [LST21])**.** Let $\overline{X} = (X_1, \ldots, X_d)$ be a tuple of sets of variables such that $|X_i| = n_i$ and let $f \in \mathbb{F}_{\mathrm{sm}}[\overline{X}]$. Let $w = (w_1, w_2, \ldots, w_d)$ be a tuple (or word) of non-zero real numbers such that $2^{|w_i|} = n_i$ for all $i \in [d]$. Corresponding to a word w, define $\mathcal{P}_w \coloneqq \{i \mid w_i > 0\}$ and $\mathcal{N}_w \coloneqq \{i \mid w_i < 0\}$. Let $\mathcal{M}_w^{\mathcal{P}}$ be the set of all set-multilinear monomials over the subset of the variable sets $X_1, X_2, \ldots, X_d$ precisely indexed by $\mathcal{P}_w$, and similarly let $\mathcal{M}_w^{\mathcal{N}}$ be the set of all set-multilinear monomials over these variable sets indexed by $\mathcal{N}_w$.

Define the 'partial derivative matrix' matrix $\mathcal{M}_w(f)$ whose rows are indexed by the elements of $\mathcal{M}_w^{\mathcal{P}}$ and columns indexed by the elements of $\mathcal{M}_w^{\mathcal{N}}$ as follows: the entry of this matrix corresponding to a row m_1 and a column m_2 is the coefficient of the monomial $m_1 \cdot m_2$ in f. We define

$$\mathrm{relrk}_w(f) \coloneqq \frac{\mathrm{rank}(\mathcal{M}_w(f))}{\sqrt{|\mathcal{M}_w^{\mathcal{P}}| \cdot |\mathcal{M}_w^{\mathcal{N}}|}} = \frac{\mathrm{rank}(\mathcal{M}_w(f))}{2^{\frac{1}{2}\sum_{i \in [d]} |w_i|}}.$$

The following is a simple result that establishes various useful properties of the relative rank measure.

Claim 1.3 ([LST21])**.**

1. (Imbalance) Say $f \in \mathbb{F}_{\mathrm{sm}}[\overline{X}(w)]$. Then, $\mathrm{relrk}_w(f) \leq 2^{-|w_{[d]}|/2}$.
2. (Sub-additivity) If $f, g \in \mathbb{F}_{\mathrm{sm}}[\overline{X}(w)]$, then $\mathrm{relrk}_w(f + g) \leq \mathrm{relrk}_w(f) + \mathrm{relrk}_w(g)$.

[2] In particular, $2^{|w_i|} \in \mathbb{N}$.

3. (Multiplicativity) Say $f = f_1 f_2 \cdots f_t$ and assume that for each $i \in [t]$, $f_i \in \mathbb{F}_{\mathrm{sm}}[\overline{X}(w|_{S_i})]$, where $(S_1, \ldots, S_t)$ is a partition of $[d]$. Then

$$\mathrm{relrk}_w(f) = \prod_{i \in [t]} \mathrm{relrk}_{w|_{S_i}}(f_i).$$

We will repeatedly make use of the following.

Theorem 1.4 (Chernoff Bound, as stated in [MU05])**.** *Suppose $X_1, \ldots, X_n$ are independent random variables taking values in $\{0, 1\}$. Let X denote their sum and let $\mu = \mathbb{E}[X]$ denote the expected value of the sum. Then for any $\delta > 0$,*

$$\begin{aligned}
\mathbb{P}[X \geq (1+\delta)\mu] &\leq \exp\left(-\frac{\delta^2 \mu}{2+\delta}\right), \text{ if } 0 \leq \delta, \\
\mathbb{P}[X \leq (1-\delta)\mu] &\leq \exp\left(-\frac{\delta^2 \mu}{2}\right), \text{ if } 0 < \delta < 1, \\
\mathbb{P}[|X - \mu| \geq \delta\mu] &\leq 2 \cdot \exp\left(-\frac{\delta^2 \mu}{3}\right), \text{ if } 0 < \delta < 1.
\end{aligned}$$

1.2 Inner Product Gadget

The following observation is used crucially to construct the hard polynomials in VP as well as VBP.

Observation 1.5 ([KS23])**.** Let $n = 2^k$ and $X_1 = \{x_{1,1}, \ldots, x_{1,n}\}$ and $X_2 = \{x_{2,1}, \ldots, x_{2,n}\}$ be two disjoint sets of variables. Then, for any *symmetric* word $w \in \{k, -k\}^2$ (i.e., where $w_1 + w_2 = 0$) and for the inner product 'gadget' $f = X_1 \cdot X_2 = \sum_{i=1}^{n} x_{1,i} x_{2,i}$, $\mathrm{relrk}_w(f) = 1$ i.e., $\mathcal{M}_w(f)$ is full-rank.

1.3 A Hard Set-Multilinear Polynomial in VBP

1.3.1 Arc-partition Measure Description

This subsection is adapted from Section 2 of [DMPY12]. Let $n = 2^k$, $d \leq n$ be an even integer, and let $X = (X_1, X_2, \ldots, X_d)$ be a collection of disjoint sets of n variables each. An *arc-partition* will be a special kind of *symmetric* word $w \in \{-k, k\}^d$ (i.e., a one-to-one map Π from X to $\{-k, k\}^d$). For the purpose of this subsection, the reader can even choose to think of the alphabet of w as $\{-1, 1\}$ (i.e., one 'positive' and one 'negative' value) – we use $k, -k$ only to remain consistent with Definition 1.2.

Identify X with the set $\{1, 2, \ldots, d\}$ in the natural way. Consider the d-cycle graph, i.e., the graph with nodes $\{1, 2, \ldots, d\}$ and edges between i and $i + 1$ modulo d. For two nodes $i \neq j$ in the d-cycle, denote by $[i, j]$ the arc between i, j, that is, the set of nodes on the path $\{i, i + 1, \ldots, j - 1, j\}$ from i to j in d-cycle. First, define a distribution $\mathcal{D}_P$ on a family of pairings (a list of disjoint pairs of nodes in the cycle) as follows. A random pairing is constructed in $d/2$ steps. At the end of step $t \in [d/2]$, we shall have a pairing $(P_1, \ldots, P_t)$ of the arc $[L_t, R_t]$. The size of $[L_t, R_t]$ is always $2t$. The first pairing contains only $P_1 = \{L_1, R_1\}$ with $L_1 = 1$ and $R_1 = 2$. Given $(P_1, \ldots, P_t)$ and $[L_t, R_t]$, define the random pair $P_t + 1$ (independently of previous choices) by

$$P_{t+1} = \begin{cases} \{L_t - 2, L_t - 1\} & \text{with probability } 1/3 \\ \{L_t - 1, R_t + 1\} & \text{with probability } 1/3 \\ \{R_t + 1, R_t + 2\} & \text{with probability } 1/3 \end{cases}$$

Define

$$[L_{t+1}, R_{t+1}] = [L_t, R_t] \cup P_{t+1}.$$

So, L_{t+1} is either $L_t - 2$, $L_t - 1$ or L_t, each value is obtained with probability $1/3$, and similarly (but not independently) for R_{t+1}.

The final pairing is $P = (P_1, P_2, \ldots, P_{d/2})$. Denote by $P \sim \mathcal{D}_P$ a pairing distributed according to $\mathcal{D}_P$.

Once a pairing P has been obtained, a word $w \in \{-k, k\}^d$ is obtained by simply randomly assigning $+k$ and $-k$ to the indices of any pair P_i. More formally, for every $t \in [d/2]$, if $P_t = \{i_t, j_t\}$, let with probability $1/2$, independently of all other choices,

$$w_{i_t} = +k \text{ and } w_{j_t} = -k,$$

and with probability $1/2$,

$$w_{i_t} = -k \text{ and } w_{j_t} = +k.$$

Denote by $w \sim \mathcal{D}$ a word in $\{-1, 1\}^n$ that is sampled using this procedure. We call such a word an *arc-partition*. For a pair $P_t = \{i_t, j_t\}$, we refer to i_t and j_t as *partners*.

Definition 1.6 (Arc-full-rank)**.** We say that a polynomial f that is set-multilinear over $X = (X_1, \ldots, X_d)$ is **arc-full-rank** if for every arc-partition $w \in \{-k, k\}^d$, $\text{relrk}_w(f) = 1$.

1.3.2 Construction of an Arc-full-rank Polynomial

Below, we describe a simple construction of a polynomial sized ABP that computes an arc-full-rank set-multilinear polynomial. The high-level idea is to construct an ABP in which every path between start-node and end-node corresponds to a specific execution of the random process which samples arc-partitions. Each node in the ABP corresponds to an arc $[L, R]$, which sends an edge to each of the nodes $[L - 2, R], [L - 1, R + 1]$ and $[L, R + 2]$. The edges have specially chosen labels that help guarantee full rank with respect to every arc-partition. For simplicity of presentation, we allow the edges of the program to be labeled by degree four set-multilinear polynomial polynomials over the corresponding subset of the variable partition. This assumption can be easily removed by replacing each edge with a polynomial-sized ABP computing the corresponding degree four polynomial.

Formally, the nodes of the program are even-size arcs in the d-cycle, d an even integer. The start-node of the program is the empty arc $\emptyset$ and the end-node is the whole cycle $[d]$ (both are "special" arcs). Let $X = (X_1, \ldots, X_d)$ be a collection of sets of variables where each $|X_i| = n$, and similarly, let $Y = (Y_1, \ldots, Y_d)$ be a distinct collection of sets of variables where each $|Y_i| = n$ (we shall refer to the Y-variables as *auxiliary* variables). For i and j in $\{1, \ldots, d\}$, let $X_i \cdot X_j$ denote the inner-product quadratic form $\sum_{k=1}^{n} x_{ik} x_{jk}$. Here, we shall assume that $X_i = \{x_{i,1}, \ldots, x_{i,n}\}$ and $Y_i = \{y_{i,1}, \ldots, y_{i,n}\}$.

Construct the branching program by connecting a node/arc of size $2t$ to three nodes/arcs of size $2t+2$. For $t = 1$, there is just one node $[1, 2]$, and the edge from start-node to it is labeled $y_{1,2}y_{2,1}(X_0 \cdot X_1)$. For $t > 1$, the node $[L, R] \supset [1, 2]$ of size $2t < d$ is connected to the three nodes: $[L-2, R], [L-1, R+1]$, and $[L, R+2]$. (It may be the case that the three nodes are the end-node.) The edge labeling is:

- The edge between $[L, R]$ and $[L-2, R]$ is labeled $y_{L-2,L-1}y_{L-1,L-2}(X_{L-2} \cdot X_{L-1})$.
- The edge between $[L, R]$ and $[L-1, R+1]$ is labeled $y_{L-1,R+1}y_{R+1,L-1}(X_{L-1} \cdot X_{R+1})$.
- The edge between $[L, R]$ and $[L, R+2]$ is labeled $y_{R+1,R+2}y_{R+2,R+1}(X_{R+1} \cdot X_{R+2})$.

Consider the ABP thus described, and the polynomial $G_{n,d}$ it computes. For every path γ from start-node to end-node in the ABP, the list of edges along γ yields a pairing P; every edge e in γ corresponds to a pair $P_e = \{i_e, j_e\}$ of nodes in d-cycle. Thus,

$$G_{n,d} = \sum_{\gamma} \prod_{e=\{i_e,j_e\}\in\gamma} y_{i_e,j_e}y_{j_e,i_e} \cdot (X_{i_e} \cdot X_{j_e}). \tag{1}$$

where the sum is over all paths γ from start-node to end-node.

Remark 1.7. There is in fact a one-to-one correspondence between pairings P and such paths γ (this follows by induction on t). Note that this is true only because pairings are tuples i.e., they are *ordered* by definition. Otherwise, it is of course still possible to obtain the same *set* of pairs in a given pairing using multiple different orderings. The sum defining $G_{n,d}$ can be thought of, therefore, as over pairings P.

The following statement summarizes the main useful property of $G_{n,d}$.

Lemma 1.8 ([KS23])**.** *Over any field $\mathbb{F}$ of characteristic zero, the polynomial $G_{n,d}$ defined above is arc-full-rank as a set-multilinear polynomial in the variables X over the field $\mathbb{F}(Y)$ of rational functions in Y.*

Proof. Let $w \sim \mathcal{D}$ be an arc-partition. We want to show that $\mathcal{M}_w(G_{n,d})$ has full rank. The arc-partition w is defined from a pairing $P = (P_1, \ldots, P_{d/2})$ (though as discussed in Remark 1.7, there could be multiple such P). The pairing P corresponds to a path γ from start-node to end-node. Consider the polynomial f that is obtained by setting every $y_{i,j} = y_{j,i} = 0$ in F such that $\{i, j\}$ is not a pair in P, and setting every $y_{i,j} = y_{j,i} = 1$ for every pair $\{i, j\}$ in P. Then, it is easy to see that the only terms that survive in Equation 1 correspond to paths (and in turn, pairings) which have the same underlying *set* of pairs as P. As a consequence, f is simply some non-zero constant times a polynomial which is full-rank (recall Observation 1.5). $M_w(f)$ being full rank then implies that $M_w(G_{n,d})$ is also full-rank. □

2 Optimal Separation between Ordered and Unordered smABPs

In this section, we prove Theorem 0.3. The result is that $G_{n,d}(X_1, \ldots, X_d, Y_1, \ldots, Y_d)$, as described in Section 1.3, does not have small *ordered* set-multilinear branching programs with respect to *any* ordering of the X_is and Y_js. More precisely, we claim that any set-multilinear branching program computing $G_{n,d}$ which is ordered with respect to some permutation of the sets in the collection $\mathcal{S} = \{X_1, \ldots, X_d, Y_1, \ldots, Y_d\}$ must have size at least $n^{\Omega(d)}$.

Proof of Theorem 0.3. Let A be a set-multilinear branching program computing $G_{n,d}$, which is ordered with respect to a permutation π of the sets in $\mathcal{S}$. That is $\pi : [2d] \to \mathcal{S}$ is a bijective map where $\pi(i) = Z$, for some $Z \in \mathcal{S}$, if all edges between layer $i-1$ and layer i are labelled using linear forms in the variables of Z. Let $\pi_X : [d] \to \{X_1, \ldots, X_d\}$ be the ordering inherited from π by the X sets, that is $\pi_X(i) = X_j$ if X_j is the i-th X-set appearing in the sequence $\pi(1), \ldots, \pi(2d)$. Define a word $w \in \{-k, k\}^d$ using π_X as follows: let

$$w_i = k \text{ if and only if } \pi_X^{-1}(X_i) \in [d/2].$$

That is, the edges labelled using linear forms in X_i appear on the 'left' half in A, when the Y sets are ignored. Note that w is *symmetric*, or in other words, $w_{[d]} = 0$.

Now, for $i \in [d]$, define $s_i(A)$ to be the size of layer $\pi^{-1}(\pi_X(i))$ in A. The technique used by Nisan [Nis91] can be used directly to show the following.

Lemma 2.1 ([Nis91])**.** *If $G_{n,d}(X_1, \ldots, X_d) \in \mathbb{K}[X_1, \ldots, X_d]$ is thought of as a polynomial only over the X variables (here $\mathbb{K} = \mathbb{F}(Y)$, the field of rational functions in Y), then $n^{d/2} \cdot \mathrm{relrk}_w(G_{n,d}) \leq s_{d/2}(A)$.*[3]

Thus, in order to prove a $n^{\Omega(d)}$ lower bound on the width of A, it suffices to lower bound $\mathrm{relrk}_w(G)$ by $n^{-\alpha d}$ for some $0 < \alpha < 1/2$. We prove such a lower bound using the following lemma.

Lemma 2.2. *Given any $w \in \{-k, k\}^d$ with $w_{[d]} = 0$, there exists an arc-partition $v \in \{-k, k\}^d$ obtained from a pairing $P = (P_1, \ldots, P_{d/2})$ such that w 'splits' a constant fraction of the pairs in P: more precisely, there is a set $S \subset P$ of size at least $d/8$ such that if $(i, j) \in S$, then $w_i + w_j = 0$.*

Let us first prove the lower bound on $\mathrm{relrk}_w(G_{n,d})$ assuming this lemma. Consider the polynomial f that is obtained by setting every $y_{i,j} = y_{j,i} = 0$ in $G_{n,d}$ such that $\{i, j\}$ is not a pair in P, and setting every $y_{i,j} = y_{j,i} = 1$ for every pair $\{i, j\}$ in P (where P is as in Lemma 2.2). Then, it is easy to see that the only terms that survive in Equation 1 correspond to paths (and in turn, pairings) which have the same underlying *set* of pairs as P. As a consequence,

$$f(X) = c_P \cdot \prod_{t=1}^{d/2} (X_{i_t} \cdot X_{j_t})$$

for a non-zero constant c_P. Next, notice that the rank of the matrix $M_w(f)$ (over the field $\mathbb{F}$) serves as a lower bound on the rank of the matrix $M_w(G_{n,d})$

[3] The $n^{d/2}$ factor comes from using relative-rank rather than rank.

(over the field $\mathbb{F}(Y)$). Indeed, if the former is r then there is a non-vanishing $r \times r$ minor of $M_w(f)$. But this implies that the determinant of the corresponding $r \times r$ sub-matrix in $M_w(G_{n,d})$ must be a non-zero polynomial in $\mathbb{F}[Y]$ as it has a non-zero evaluation. From this we conclude that $M_w(G_{n,d})$ also has a non-vanishing $r \times r$ minor and therefore has rank at least r. Stated in terms of relative rank, we have showed that $\mathrm{relrk}_w(f) \leq \mathrm{relrk}_w(G_{n,d})$.

Now, given the set $S \subset [d/2]$ from Lemma 2.2, we can lower bound the relative rank of $M_w(f)$ as follows: first, note that if for some $t \in [d/2]$, the pair P_t of P is in S, then by Observation 1.5, $\mathrm{relrk}_{w|_{P_t}}(X_{i_t} \cdot X_{j_t}) = 1$. Secondly, if w does *not* split some pair P_t of P (i.e., $w_{i_t} = w_{j_t}$), then $M_{w|_{P_t}}(X_{i_t} \cdot X_{j_t})$ is simply a one-dimensional vector (either $1 \times n^2$ or $n^2 \times 1$) and we trivially have $\mathrm{relrk}_{w|_{P_t}}(X_{i_t} \cdot X_{j_t}) = 1/n$. Combining, and using the multiplicativity of $\mathrm{relrk}_w(\cdot)$ (third item of Claim 1.3),

$$\begin{aligned}\mathrm{relrk}_w(f(X)) = \mathrm{relrk}_w(f(X)/c_P) &= \prod_{t=1}^{d/2} \mathrm{relrk}_{w|_{P_t}}(X_{i_t} \cdot X_{j_t}) \\ &= \prod_{t \in S} \underbrace{\mathrm{relrk}_{w|_{P_t}}(X_{i_t} \cdot X_{j_t})}_{=1} \prod_{t \in [d/2] \setminus S} \underbrace{\mathrm{relrk}_{w|_{P_t}}(X_{i_t} \cdot X_{j_t})}_{\geq 1/n} \geq \frac{1}{n^{d/2-|S|}}.\end{aligned}$$

Therefore, we have $\mathrm{relrk}_w(G_{n,d}) \geq \mathrm{relrk}_w(f) \geq 1/n^{d/2-|S|}$ and hence, by Lemma 2.1, the size of A is at least $s_{d/2}(A) \geq n^{d/2} \cdot \mathrm{relrk}_w(G_{n,d}) \geq n^{|S|} = n^{\Omega(d)}$. □

We now give the proof of Lemma 2.2.

Proof of Lemma 2.2. Let $L_+, L_-, R_+, R_- \in [d]$ be defined as follows.

$$R_+ = \left\{ i \in \left[3, \frac{d}{2}+1\right] \quad : \quad w_i > 0 \right\} \qquad R_- = \left\{ i \in \left[3, \frac{d}{2}+1\right] \quad : \quad w_i < 0 \right\}$$

$$L_+ = \left\{ i \in \left[\frac{d}{2}+2, d\right] \quad : \quad w_i > 0 \right\} \qquad L_- = \left\{ i \in \left[\frac{d}{2}+2, d\right] \quad : \quad w_i < 0 \right\}$$

Clearly, either $|R_+| \geq \frac{d}{4}$ or $|R_-| \geq \frac{d}{4}$. Without loss of generality, let us assume that $|R_+| \geq \frac{d}{4}$.

Also, without loss of generality[4] we can assume that $w_1 + w_2 = 0$. Then $|R_+| = |L_-|$, and say $|R_+| = p$. Further, let

$$R_+ = \{i_1, \ldots, i_p\} \text{ with } i_1 < \cdots < i_p \quad \text{and} \quad L_- = \{j_1, \ldots, j_p\} \text{ with } j_1 > \cdots > j_p.$$

We can then define an initial pairing, $P^0 = \{(i_\ell, j_\ell)\}_{\ell \in [p]}$. Let us also define the set $S^0 = \emptyset$. The goal is to iteratively update P^0 such that, at the end, we have a pairing corresponding to an arc-partition. We will also update S^0 at each step so that, at the end, each pair in S are of opposite signs.

Let $u_0 = 2, v_0 = 1$. Intuitively, u_ℓ, v_ℓ are the right most and left-most points, respectively, of the partial arc-partition, P^ℓ, defined till the ℓ-th iteration. Given $u_{\ell-1}, v_{\ell-1}, P^{\ell-1}, S^{\ell-1}$ for any $\ell \in [p]$, we will define $u_\ell, v_\ell, P^\ell, S^\ell$ as follows. Note that the calculations are $\pmod d$ when $\ell = 1$.

[4] If this is not the case, then $|L_-| - 2 \leq |R_+| \leq |L_-| + 2$ and we redefine R_+, L_- to be the largest possible subsets of the originally defined R_+, L_-, respectively, such that $|R_+| = |L_-|$.

Case 1: $(i_\ell - u_{\ell-1})$ and $(v_{\ell-1} - j_\ell)$ are both odd.

$$P^\ell = P^{\ell-1} \cup \{(u_{\ell-1}+1, u_{\ell-1}+2), \ldots, (i_\ell-2, i_\ell-1)\} \cup \{(v_{\ell-1}-2, v_{\ell-1}-1), \ldots, (j_\ell+1, j_\ell+2)\}$$

$$u_\ell = i_\ell \qquad v_\ell = j_\ell \qquad \text{and} \qquad S^\ell = S^{\ell-1} \cup \{(i_\ell, j_\ell)\}$$

Case 2: $(i_\ell - u_{\ell-1})$ is even and $(v_{\ell-1} - j_\ell)$ is odd.

We first define

$$Q^\ell = \begin{cases} (P^{\ell-1} \setminus \{(i_\ell, j_\ell)\}) \cup \{(u_{\ell-1}+1, u_{\ell-1}+2), \ldots, (i_\ell - 1, i_\ell)\} & \\ \qquad\qquad\qquad\qquad \cup \{(v_{\ell-1}-2, v_{\ell-1}-1), \ldots, (j_\ell+1, j_\ell+2\}, & \text{if } (i_\ell - u_{\ell-1}) > 0 \\ (P^{\ell-1} \setminus \{(i_\ell, j_\ell)\}) \cup \{(v_{\ell-1}-2, v_{\ell-1}-1), \ldots, (j_\ell+1, j_\ell+2)\} & \text{if } (i_\ell - u_{\ell-1}) = 0, \end{cases}$$

and then define

$$P^\ell = \begin{cases} Q^\ell \cup \{(j_\ell - 1, j_\ell)\} & \text{if} \quad j_\ell - 1 \in L_+. \\ Q^\ell \cup \{(i_\ell + 1, j_\ell)\} & \text{otherwise.} \end{cases}$$

Also,

$$u_\ell = \begin{cases} i_\ell & \text{if} \quad j_\ell - 1 \in L_+ \\ i_\ell + 1 & \text{otherwise.} \end{cases} \qquad v_\ell = \begin{cases} j_\ell - 1 & \text{if} \quad j_\ell - 1 \in L_+ \\ j_\ell & \text{otherwise.} \end{cases}$$

and

$$S^\ell = \begin{cases} S^{\ell-1} \cup \{(i_\ell, j_{\ell-1})\} & \text{if } (i_\ell - u_{\ell-1}) = 0 \\ S^{\ell-1} \cup \{(i_\ell - 1, i_\ell)\} & \text{otherwise.} \end{cases}$$

Case 3: $(i_\ell - u_{\ell-1})$ is odd and $(v_{\ell-1} - j_\ell)$ is even.

We first define

$$Q^\ell = \begin{cases} (P^{\ell-1} \setminus \{(i_\ell, j_\ell)\}) \cup \{(u_{\ell-1}+1, u_{\ell-1}+2), \ldots, (i_\ell - 2, i_\ell - 1)\} & \\ \qquad\qquad\qquad\qquad \cup \{(v_{\ell-1}-2, v_{\ell-1}-1), \ldots, (j_\ell, j_\ell+1)\} & \text{if } (v_{\ell-1} - j_\ell) > 0, \\ (P^{\ell-1} \setminus \{(i_\ell, j_\ell)\}) \cup \{(u_{\ell-1}+1, u_{\ell-1}+2), \ldots, (i_\ell - 2, i_\ell - 1)\} & \text{if } (v_{\ell-1} - j_\ell) = 0, \end{cases}$$

and then define

$$P^\ell = \begin{cases} Q^\ell \cup \{(i_\ell, i_\ell + 1)\} & \text{if} \quad i_\ell + 1 \in R_-. \\ Q^\ell \cup \{(i_\ell, j_\ell - 1)\} & \text{otherwise.} \end{cases}$$

Also,

$$u_\ell = \begin{cases} i_\ell + 1 & \text{if} \quad i_\ell + 1 \in R_- \\ i_\ell & \text{otherwise.} \end{cases} \qquad v_\ell = \begin{cases} j_\ell & \text{if} \quad i_\ell + 1 \in R_- \\ j_\ell - 1 & \text{otherwise.} \end{cases}$$

and

$$S^\ell = \begin{cases} S^{\ell-1} \cup \{(i_{\ell-1}, j_\ell)\} & \text{if } (v_{\ell-1} - j_\ell) = 0. \\ S^{\ell-1} \cup \{(j_\ell, j_\ell + 1)\} & \text{otherwise.} \end{cases}$$

Case 4: $(i_\ell - u_{\ell-1})$ and $(v_{\ell-1} - j_\ell)$ are both even.

We first define

$$Q^\ell = \begin{cases} (P^{\ell-1} \setminus \{(i_\ell, j_\ell)\}) & \text{if } (i_\ell - u_{\ell-1}) = 0 \\ (P^{\ell-1} \setminus \{(i_\ell, j_\ell)\}) \cup \{(u_{\ell-1}+1, u_{\ell-1}+2), \ldots, (i_\ell - 1, i_\ell)\} & \text{otherwise} \end{cases}$$

and then define

$$P^\ell = \begin{cases} Q^\ell & \text{if } (v_{\ell-1} - j_\ell) = 0 \\ Q^\ell \cup \{(v_{\ell-1}-2, v_{\ell-1}-1), \ldots, (j_\ell, j_\ell+1)\} & \text{otherwise.} \end{cases}$$

Also,

$$u_\ell = i_\ell \qquad v_\ell = j_\ell \qquad \text{and} \qquad S^\ell = \begin{cases} S^{\ell-1} \cup \{(i_\ell, j_{\ell-1})\} & \text{if } (i_\ell - u_{\ell-1}) = 0. \\ S^{\ell-1} \cup \{(i_\ell - 1, i_\ell)\} & \text{otherwise.} \end{cases}$$

Note that $(v_p - u_p)$ must be odd. So finally, we define $P = P^p \cup \{(u_p + 1, u_p + 2), \ldots, (v_p - 2, v_p - 1)\}$ and $S = S^p$. Firstly note that P corresponds to an arc-partition, since P^0 is a partial arc-partition and it is easy to check that for each $\ell \in [p]$, P^ℓ is a valid extension of $P^{\ell-1}$.

Since $p \geq d/8$, S clearly has size at least $d/8$. The only thing left to complete the proof is to check that S has the other required property. For any $\ell \in [p]$, let us assume that $S^{\ell-1} \subset P$ has the property that if $(i, j) \in S^{\ell-1}$, $w_i + w_j = 0$. In Case 1, clearly, S^ℓ continues to have this property. In Cases 2 and 4, if $(i_\ell - u_{\ell-1}) > 0$, then again S^ℓ clearly continues to have this property. When $(i_\ell - u_{\ell-1}) = 0$, it must mean that $i_\ell = u_{\ell-1} = i_{\ell-1} + 1$ and so $i_{\ell-1} + 1 \notin R_-$. That is, in the previous iteration Case 2 had been true and $(i_\ell, j_{\ell-1}) = (i_{\ell-1} + 1, j_{\ell-1})$ had been added to $P^{\ell-1}$. Therefore $S^\ell \subset P$ and, also, $w_{i_\ell} + w_{j_{\ell-1}} = 0$. A similar argument for Case 3 then completes the proof. □

References

[AFS+18] Matthew Anderson, Michael A. Forbes, Ramprasad Saptharishi, Amir Shpilka, and Ben Lee Volk. Identity testing and lower bounds for read-k oblivious algebraic branching programs. *ACM Trans. Comput. Theory*, 10(1):3:1–3:30, 2018.

[AGKS15] Manindra Agrawal, Rohit Gurjar, Arpita Korwar, and Nitin Saxena. Hitting-sets for ROABP and sum of set-multilinear circuits. *SIAM J. Comput.*, 44(3):669–697, 2015.

[AR16] Vikraman Arvind and S. Raja. Some lower bound results for set-multilinear arithmetic computations. *Chic. J. Theor. Comput. Sci.*, 2016, 2016.

[BDS22] C. S. Bhargav, Sagnik Dutta, and Nitin Saxena. Improved lower bound, and proof barrier, for constant depth algebraic circuits. In Stefan Szeider, Robert Ganian, and Alexandra Silva, editors, *47th International Symposium on Mathematical Foundations of Computer Science, MFCS 2022, August 22-26, 2022, Vienna, Austria*, volume 241 of *LIPIcs*, pages 18:1–18:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[BDS24] C.S. Bhargav, Prateek Dwivedi, and Nitin Saxena. Lower bounds for the sum of small-size algebraic branching programs. *To appear in the proceedings of the Annual Conference on Theory and Applications of Models of Computation*, 2024. `https://www.cse.iitk.ac.in/users/nitin/papers/sumRO.pdf`.

[BG22] Vishwas Bhargava and Sumanta Ghosh. Improved hitting set for orbit of roabps. *Comput. Complex.*, 31(2):15, 2022.

[BS21] Pranav Bisht and Nitin Saxena. Blackbox identity testing for sum of special roabps and its border class. *Comput. Complex.*, 30(1):8, 2021.

[Bür00] Peter Bürgisser. Cook's versus valiant's hypothesis. *Theor. Comput. Sci.*, 235(1):71–88, 2000.

[CKSV22] Prerona Chatterjee, Mrinal Kumar, Adrian She, and Ben Lee Volk. Quadratic lower bounds for algebraic branching programs and formulas. *Comput. Complex.*, 31(2):8, 2022.

[DMPY12] Zeev Dvir, Guillaume Malod, Sylvain Perifel, and Amir Yehudayoff. Separating multilinear branching programs and formulas. In Howard J. Karloff and Toniann Pitassi, editors, *Proceedings of the 44th Symposium on Theory of Computing Conference, STOC 2012, New York, NY, USA, May 19 - 22, 2012*, pages 615–624. ACM, 2012.

[FS13] Michael A. Forbes and Amir Shpilka. Quasipolynomial-time identity testing of non-commutative and read-once oblivious algebraic branching programs. In *54th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2013, 26-29 October, 2013, Berkeley, CA, USA*, pages 243–252. IEEE Computer Society, 2013.

[FSS14] Michael A. Forbes, Ramprasad Saptharishi, and Amir Shpilka. Hitting sets for multilinear read-once algebraic branching programs, in any order. In David B. Shmoys, editor, *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*, pages 867–875. ACM, 2014.

[GG20] Zeyu Guo and Rohit Gurjar. Improved explicit hitting-sets for roabps. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.

[GKS17] Rohit Gurjar, Arpita Korwar, and Nitin Saxena. Identity testing for constant-width, and any-order, read-once oblivious arithmetic branching programs. *Theory Comput.*, 13(1):1–21, 2017.

[GKST17] Rohit Gurjar, Arpita Korwar, Nitin Saxena, and Thomas Thierauf. Deterministic identity testing for sum of read-once oblivious arithmetic branching programs. *Comput. Complex.*, 26(4):835–880, 2017.

[GR21] Purnata Ghosal and B. V. Raghavendra Rao. Limitations of sums of bounded read formulas and abps. In Rahul Santhanam and Daniil Musatov, editors, *Computer Science - Theory and Applications - 16th International Computer Science Symposium in Russia, CSR 2021, Sochi, Russia, June 28 - July 2, 2021, Proceedings*, volume 12730 of *Lecture Notes in Computer Science*, pages 147–169. Springer, 2021.

[GV20] Rohit Gurjar and Ben Lee Volk. Pseudorandom bits for oblivious branching programs. *ACM Trans. Comput. Theory*, 12(2):8:1–8:12, 2020.

[Jan08] Maurice J. Jansen. Lower bounds for syntactically multilinear algebraic branching programs. In Edward Ochmanski and Jerzy Tyszkiewicz, editors, *Mathematical Foundations of Computer Science 2008, 33rd International Symposium, MFCS 2008, Torun, Poland, August 25-29, 2008, Proceedings*, volume 5162 of *Lecture Notes in Computer Science*, pages 407–418. Springer, 2008.

[Kay12] Neeraj Kayal. An exponential lower bound for the sum of powers of bounded degree polynomials. *Electron. Colloquium Comput. Complex.*, TR12-081, 2012.

[KLSS17] Neeraj Kayal, Nutan Limaye, Chandan Saha, and Srikanth Srinivasan. An exponential lower bound for homogeneous depth four arithmetic formulas. *SIAM J. Comput.*, 46(1):307–335, 2017.

[KNS20] Neeraj Kayal, Vineet Nair, and Chandan Saha. Separation between read-once oblivious algebraic branching programs (roabps) and multilinear depth-three circuits. *ACM Trans. Comput. Theory*, 12(1):2:1–2:27, 2020.

[KS17] Mrinal Kumar and Shubhangi Saraf. On the power of homogeneous depth 4 arithmetic circuits. *SIAM J. Comput.*, 46(1):336–387, 2017.

[KS22] Deepanshu Kush and Shubhangi Saraf. Improved low-depth set-multilinear circuit lower bounds. In Shachar Lovett, editor, *37th Computational Complexity Conference, CCC 2022, July 20-23, 2022, Philadelphia, PA, USA*, volume 234 of *LIPIcs*, pages 38:1–38:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[KS23] Deepanshu Kush and Shubhangi Saraf. Near-optimal set-multilinear formula lower bounds. In Amnon Ta-Shma, editor, *38th Computational Complexity Conference, CCC 2023, July 17-20, 2023, Warwick, UK*, volume 264 of *LIPIcs*, pages 15:1–15:33. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

[KSS14] Neeraj Kayal, Chandan Saha, and Ramprasad Saptharishi. A super-polynomial lower bound for regular arithmetic formulas. In David B. Shmoys, editor, *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*, pages 146–153. ACM, 2014.

[KST16] Neeraj Kayal, Chandan Saha, and Sébastien Tavenas. An almost cubic lower bound for depth three arithmetic circuits. In Ioannis Chatzigiannakis, Michael Mitzenmacher, Yuval Rabani, and Davide Sangiorgi, editors, *43rd International Colloquium on Automata, Languages, and Programming, ICALP 2016, July 11-15, 2016, Rome, Italy*, volume 55 of *LIPIcs*, pages 33:1–33:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016.

[LST21] Nutan Limaye, Srikanth Srinivasan, and Sébastien Tavenas. Superpolynomial lower bounds against low-depth algebraic circuits. In *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*, pages 804–814. IEEE, 2021.

[LST22] Nutan Limaye, Srikanth Srinivasan, and Sébastien Tavenas. On the partial derivative method applied to lopsided set-multilinear polynomials. In Shachar Lovett, editor, *37th Computational Complexity Conference, CCC 2022, July 20-23, 2022, Philadelphia, PA, USA*, volume 234 of *LIPIcs*, pages 32:1–32:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[MU05] Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.

[Nis91] Noam Nisan. Lower bounds for non-commutative computation (extended abstract). In Cris Koutsougeras and Jeffrey Scott Vitter, editors, *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing, May 5-8, 1991, New Orleans, Louisiana, USA*, pages 410–418. ACM, 1991.

[NW97] Noam Nisan and Avi Wigderson. Lower bounds on arithmetic circuits via partial derivatives. *Comput. Complex.*, 6(3):217–234, 1997.

[Raz06] Ran Raz. Separation of multilinear circuit and formula size. *Theory Comput.*, 2(6):121–135, 2006.

[Raz13] Ran Raz. Tensor-rank and lower bounds for arithmetic formulas. *J. ACM*, 60(6):40:1–40:15, 2013.

[RR20] C. Ramya and B. V. Raghavendra Rao. Lower bounds for special cases of syntactic multilinear abps. *Theor. Comput. Sci.*, 809:1–20, 2020.

[RS05] Ran Raz and Amir Shpilka. Deterministic polynomial identity testing in non-commutative models. *Comput. Complex.*, 14(1):1–19, 2005.

[RY08] Ran Raz and Amir Yehudayoff. Balancing syntactically multilinear arithmetic circuits. *Comput. Complex.*, 17(4):515–535, 2008.

[RY09] Ran Raz and Amir Yehudayoff. Lower bounds and separations for constant depth multilinear circuits. *Comput. Complex.*, 18(2):171–207, 2009.

[Sap15] Ramprasad Saptharishi. A survey of lower bounds in arithmetic circuit complexity. *Github Survey*, 2015.

[SY10] Amir Shpilka and Amir Yehudayoff. Arithmetic circuits: A survey of recent results and open questions. *Found. Trends Theor. Comput. Sci.*, 5(3-4):207–388, 2010.

[TLS22] Sébastien Tavenas, Nutan Limaye, and Srikanth Srinivasan. Set-multilinear and non-commutative formula lower bounds for iterated matrix multiplication. In Stefano Leonardi and Anupam Gupta, editors, *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing, Rome, Italy, June 20 - 24, 2022*, pages 416–425. ACM, 2022.

[Val79] L. G. Valiant. Completeness classes in algebra. In *Proceedings of the Eleventh Annual ACM Symposium on Theory of Computing*, STOC '79, page 249–261, New York, NY, USA, 1979. Association for Computing Machinery.